Chapter 4: Threads & Concurrency

these slides are edited and some of the contents are taken from
https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf
and
http://www.it.uu.se/education/course/homepage/os/vt18/module-4/implementing-threads/

Main two point:

1) threads vs. process

2) kernel-level vs. user-level threads

Review of process

Concurrency and threads

- Overview
- Multicore Programming

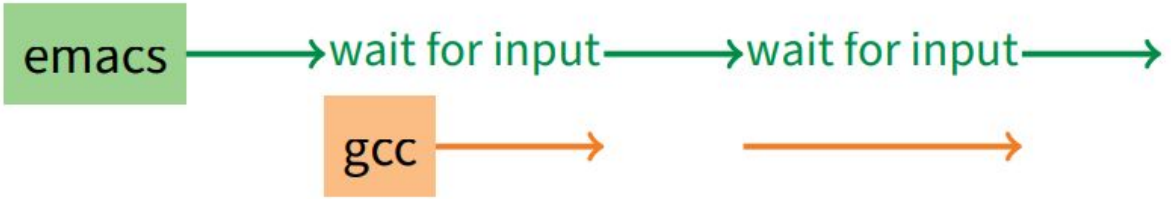
Review: Processes

- A process is an instance of a program running

- Why processes?
  - Simplicity of programming
  - Speed: Higher throughput, lower latency

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Speed



Multiple processes can increase **CPU utilization**

Multiple processes can reduce **latency**

Running A then B requires 100 sec for B to complete

A is slower than if it had whole machine to itself,
but still <= 100 sec unless both A and B completely CPU-bound

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Multitasking in real world

- 1 worker 10 months to make 1 widget
- hire 100 workers to make 100 widgets
  - Latency for first widget >> 1/10 month
    - setup/learning etc
  - Throughput may be < 10 widgets per month
    - **if can't perfectly parallelize task**

- Or 100 workers making 10,000 widgets may achieve > 10 widgets/month
  - **if workers never idly wait**
    - **capitalism at its best**😔

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

A **process's view** of the world

Each process has own view of machine

- Its own address space – *(char *)0xc000 different in P1 & P2
- Its own open files
- Its own virtual CPU (through preemptive
- multitasking)

Simplifies programming model

- gcc does not care that firefox is running

Sometimes want interaction between processes

- Simplest is through files: emacs edits file, gcc compiles it
- More complicated: Shell/command, Window manager/app.

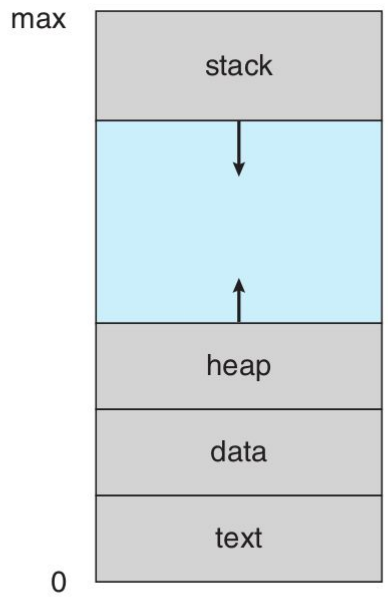
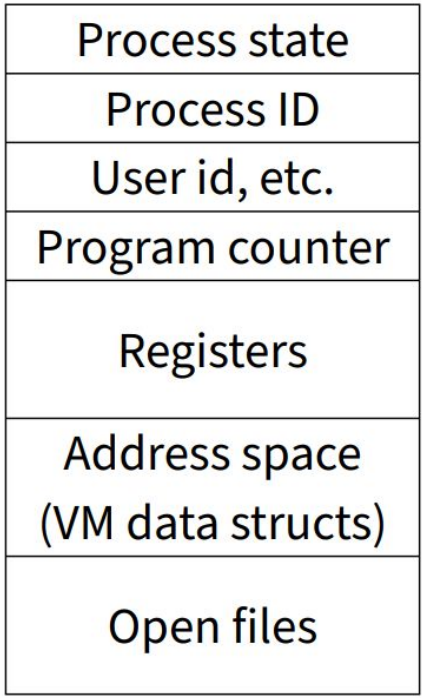


Figure 3.1 Layout of a process in memory.

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Kernel's view of processes: implementing a process

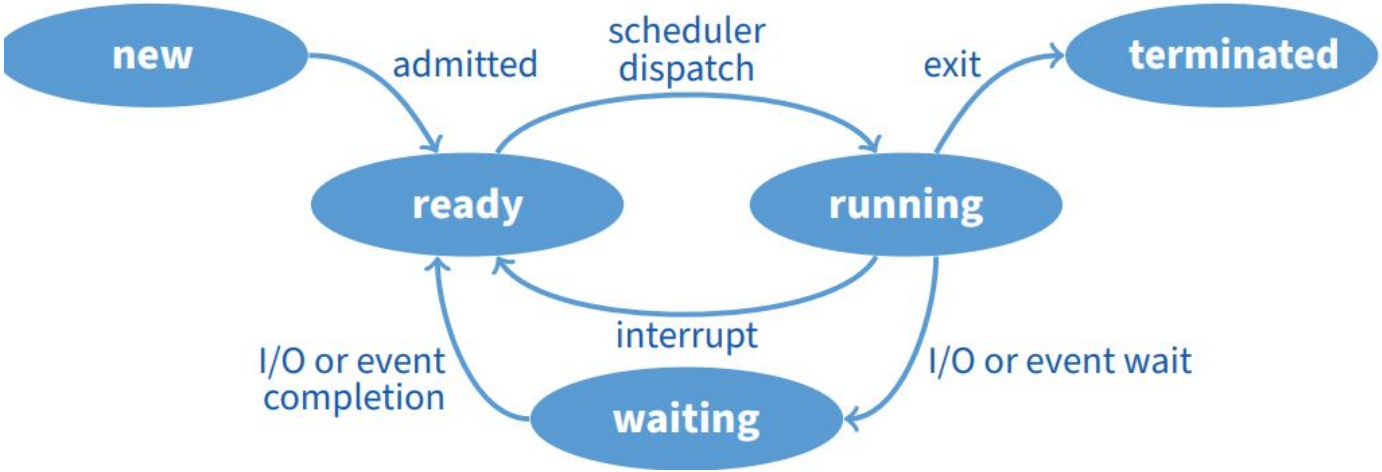
- Keep a data structure for each process
  - Process Control Block (PCB)
  - Called proc in Unix, task_struct in Linux,
- Tracks state of the process
  - Running, ready (runnable), waiting, etc.
- Includes information necessary to run
  - Registers, virtual memory mappings, etc.
  - Open files (including memory mapped files)
- Various other data about the process
  - Credentials (user/group ID), signal mask, controlling terminal, priority, accounting statistics, whether being debugged, which system call binary emulation in use, . . .



PCB

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Process states



Process can be in one of several states

- new & terminated at beginning & end of life
- running – currently executing (or will execute on kernel return)
- ready – can run, but kernel has chosen different process to run
- waiting – needs async event (e.g., disk operation) to proceed

Which process should kernel run?

- if 0 runnable, run idle loop (or halt CPU), if 1 runnable, run it
- if >1 runnable, must make scheduling decision

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Preemption

Can preempt a process **when kernel gets control**

- **Running process can vector control to kernel**
  - System call, page fault, illegal instruction, etc.
  - May put current process to sleep
    - e.g., read from disk
  - May make other process runnable
    - e.g., fork, write to pipe
- **Periodic timer interrupt**
  - If running process used up quantum, schedule another
- **Device interrupt**
  - Disk request completed, or packet arrived on network
  - Previously waiting process becomes runnable
  - Schedule if higher priority than current running proc.

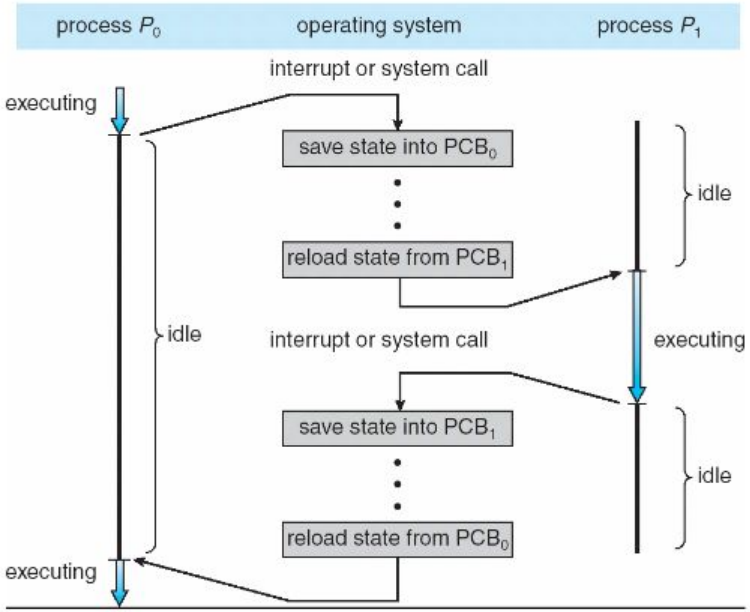
Changing running process is called **a context switch**

https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Context switch

Very machine dependent. Typical things include:

- Save program counter and integer registers (always)
- Save floating point or other special registers
- Save condition codes
- Change virtual address translations

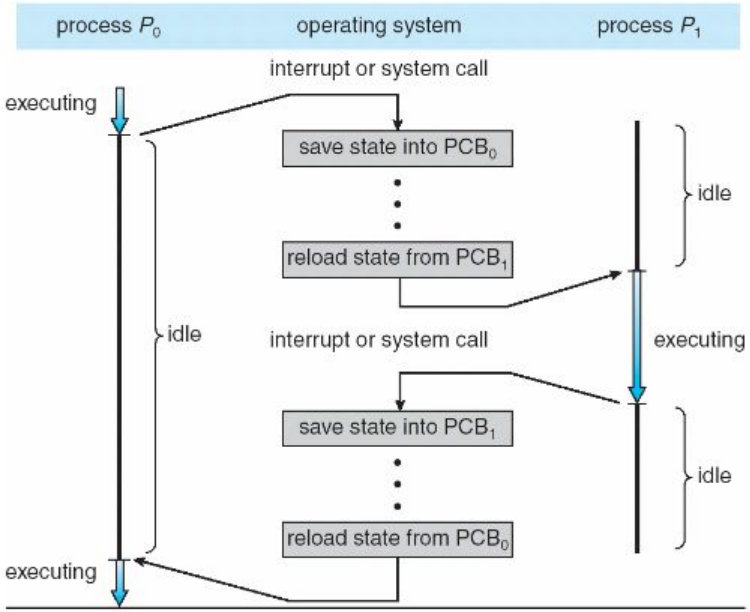


https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Context switch: Cost

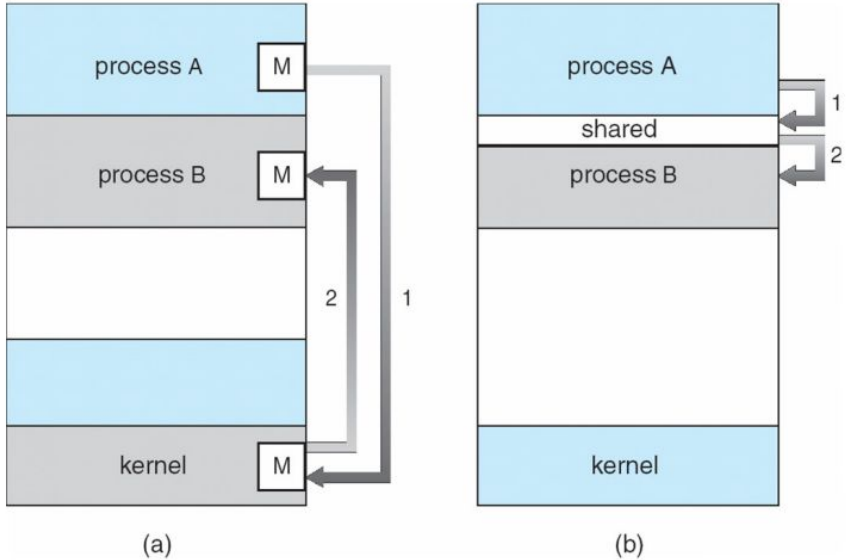
Non-negligible cost

- Save/restore floating point registers expensive
  - Optimization: only save if process used floating point
- May require **flushing TLB** (memory translation hardware)
- Usually causes more cache misses (switch working sets)



https://www.scs.stanford.edu/24wi-cs212/notes/processes.pdf

Inter-process communication



- How can processes interact in real time?
  - (a) By passing messages through the kernel
  - (b) By sharing a region of physical memory
  - (c) Through asynchronous signals or alerts

Creating/deleting processes in Unix

- `int fork (void);`
  - Create new process that is exact copy of current one
  - Returns *process ID* of new process in "parent"
  - Returns 0 in "child"
- `int waitpid (int pid, int *stat, int opt);`
  - `pid` – process to wait for, or -1 for any
  - `stat` – will contain exit value, or signal
  - `opt` – usually 0 or `WNOHANG`
  - Returns process ID or -1 on error
- `void exit (int status);`
  - Current process ceases to exist
  - `status` shows up in `waitpid` (shifted)
  - By convention, `status` of 0 is success, non-zero error
- `int kill (int pid, int sig);`
  - Sends signal `sig` to process `pid`
  - `SIGTERM` most common value, kills process by default (but application can catch it for "cleanup")
  - `SIGKILL` stronger, kills process always

Other examples

E.g. windows system call [CreateProcessA function (processthreadsapi.h) - Win32 apps | Microsoft Learn]

[CreateProcessAsUserA function (processthreadsapi.h) - Win32 apps | Microsoft Learn]

```
BOOL CreateProcessAsUserA(
  [in, optional]      HANDLE                hToken,
  [in, optional]      LPCSTR                lpApplicationName,
  [in, out, optional] LPSTR                 lpCommandLine,
  [in, optional]      LPSECURITY_ATTRIBUTES lpProcessAttributes,
  [in, optional]      LPSECURITY_ATTRIBUTES lpThreadAttributes,
  [in]                BOOL                  bInheritHandles,
  [in]                DWORD                 dwCreationFlags,
  [in, optional]      LPVOID                lpEnvironment,
  [in, optional]      LPCSTR                lpCurrentDirectory,
  [in]                LPSTARTUPINFOA        lpStartupInfo,
  [out]               LPPROCESS_INFORMATION lpProcessInformation
);
```

Running programs

- `int execve (char *prog, char **argv, char **envp);`
  - `prog` – full pathname of program to run
  - `argv` – argument vector that gets passed to `main`
  - `envp` – environment variables, e.g., `PATH`, `HOME`
- **Generally called through a wrapper functions**
  - `int execvp (char *prog, char **argv);`
    Search `PATH` for prog, use current environment
  - `int execlp (char *prog, char *arg, ...);`
    List arguments one at a time, finish with `NULL`

```
pid_t pid; char **av;
void doexec () {
  execvp (av[0], av);
  perror (av[0]);
  exit (1);
}
```

```
/* ... main loop: */
for (;;) {
  parse_next_line_of_input (&av, stdin);
  switch (pid = fork ()) {
  case -1:
    perror ("fork"); break;
  case 0:
    doexec ();
  default:
    waitpid (pid, NULL, 0); break;
  }
}
```

- `int dup2 (int oldfd, int newfd);`
  - Closes `newfd`, if it was a valid descriptor
  - Makes `newfd` an exact copy of `oldfd`
  - Two file descriptors will share same offset (`lseek` on one will affect both)
- `int fcntl (int fd, int cmd, ...)` – **misc fd configuration**
  - `fcntl (fd, F_SETFD, val)` – sets close-on-exec flag
    When `val == 0`, `fd` not inherited by spawned programs
  - `fcntl (fd, F_GETFL)` – get misc fd flags
  - `fcntl (fd, F_SETFL, val)` – set misc fd flags

Loop that reads a command and executes it

Recognizes

`command < input > output 2> errlog`

```
void doexec (void) {
  int fd;
  if (infile) {     /* non-NULL for "command < infile" */
    if ((fd = open (infile, O_RDONLY)) < 0) {
      perror (infile);
      exit (1);
    }
    if (fd != 0) {
      dup2 (fd, 0);
      close (fd);
    }
  }

  /* ... do same for outfile→fd 1, errfile→fd 2 ... */

  execvp (av[0], av);
  perror (av[0]);
  exit (1);
}
```

Example IPC: Pipes-message passing through kernel

- `int pipe (int fds[2]);`
  - Returns two file descriptors in `fds[0]` and `fds[1]`
  - Data written to `fds[1]` will be returned by `read` on `fds[0]`
  - When last copy of `fds[1]` closed, `fds[0]` will return EOF
  - Returns 0 on success, -1 on error
- **Operations on pipes**
  - `read`/`write`/`close` – as with files
  - When `fds[1]` closed, `read(fds[0])` returns 0 bytes
  - When `fds[0]` closed, `write(fds[1])`:
    - Kills process with `SIGPIPE`
    - Or if signal ignored, fails with EPIPE

```
void doexec (void) {
  while (outcmd) {
    int pipefds[2]; pipe (pipefds);
    switch (fork ()) {
    case -1:
      perror ("fork"); exit (1);
    case 0:
      dup2 (pipefds[1], 1);
      close (pipefds[0]); close (pipefds[1]);
      outcmd = NULL;
      break;
    default:
      dup2 (pipefds[0], 0);
      close (pipefds[0]); close (pipefds[1]);
      parse_command_line (&av, &outcmd, outcmd);
      break;
    }
  }
```

Overview of multicore programming

- Overview
- Multicore Programming
- Multithreading Models

Multicore Programming

- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**

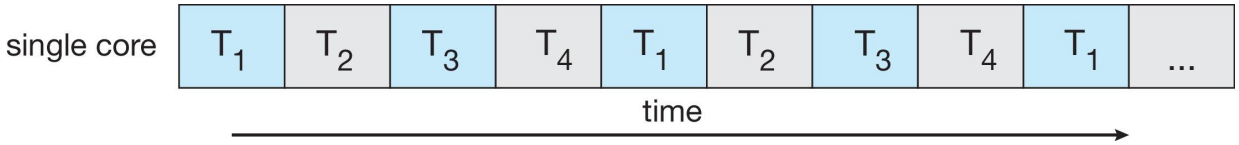
Multicore Programming: key definitions

- ***Parallelism*** implies a system can perform more than one task simultaneously

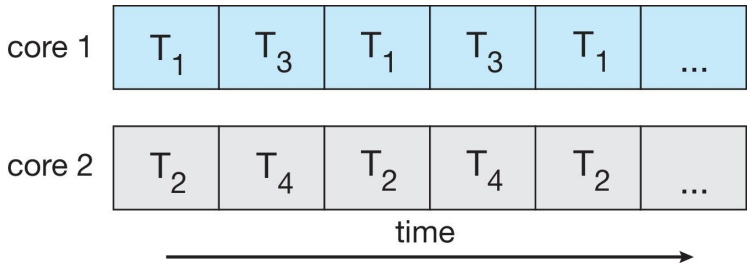
- Operations are **concurrent** if they can progress independently at the same time
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

Concurrency (think as logical) vs. Parallelism (actual)

- **Concurrent execution on single-core system:**



- **Parallelism on a multi-core system:**



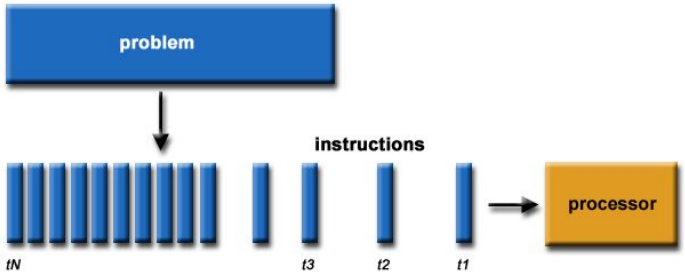
Serial vs Parallel computing

A process is working on multiple tasks at the same time

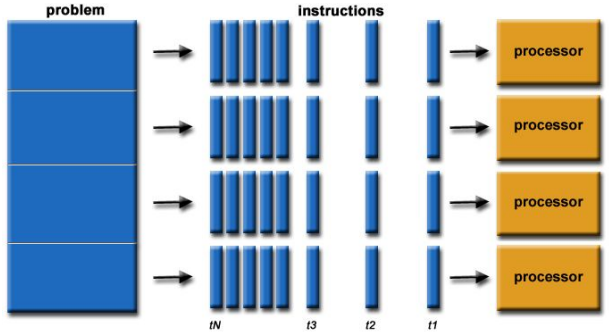
Parallelism: splitting tasks into subtasks

Parallel computing: running subtasks (concurrent parts) parallel in multiple processor

Serial computing

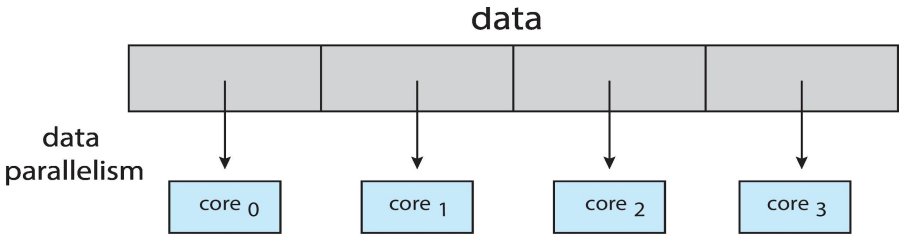


Parallel computing

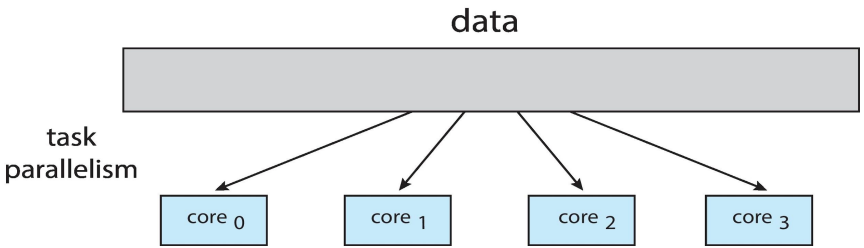


Multicore Programming: Types of parallelism

- **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each



- **Task parallelism** – distributes threads across cores, each thread performing unique operation



Matrix addition: Concurrent parts?

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

$$\mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

$$\mathbf{A} + \mathbf{B} = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & \cdots & a_{1n} + b_{1n} \\ a_{21} + b_{21} & a_{22} + b_{22} & \cdots & a_{2n} + b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} + b_{m1} & a_{m2} + b_{m2} & \cdots & a_{mn} + b_{mn} \end{bmatrix}$$

Data: **a_ij, b_ij**

Task: Addition

The same task, different data!

Matrix multiplication: Concurrent parts?

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \qquad \mathbf{B} = \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{bmatrix}$$

$$\mathbf{C} = \mathbf{AB}$$

$$\mathbf{C} = \begin{pmatrix} a_{11}b_{11} + \cdots + a_{1n}b_{n1} & a_{11}b_{12} + \cdots + a_{1n}b_{n2} & \cdots & a_{11}b_{1p} + \cdots + a_{1n}b_{np} \\ a_{21}b_{11} + \cdots + a_{2n}b_{n1} & a_{21}b_{12} + \cdots + a_{2n}b_{n2} & \cdots & a_{21}b_{1p} + \cdots + a_{2n}b_{np} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1}b_{11} + \cdots + a_{mn}b_{n1} & a_{m1}b_{12} + \cdots + a_{mn}b_{n2} & \cdots & a_{m1}b_{1p} + \cdots + a_{mn}b_{np} \end{pmatrix}$$

Data: **a_i, B** or **A, bj**

Task: Multiplication(and addition)

The same task, different data!

What if we want to do both?

Matrix addition

$\mathbf{T = A + B}$

Matrix multiplication

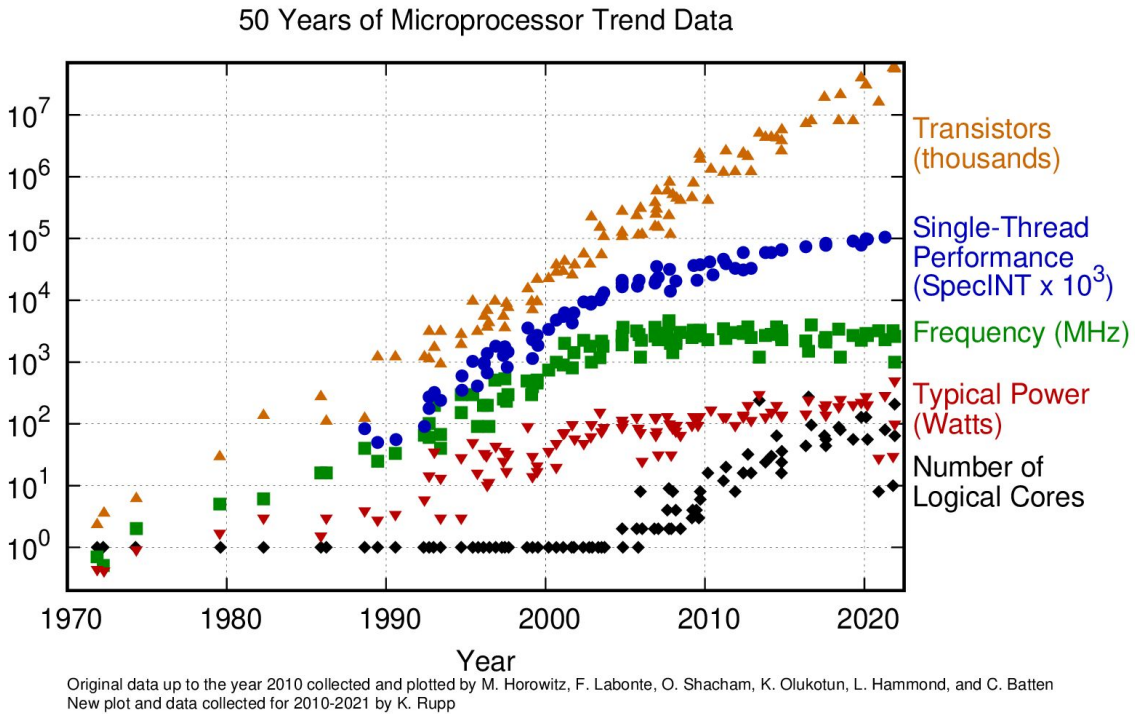
$\mathbf{C = AB}$

Data: $\mathbf{A, B}$

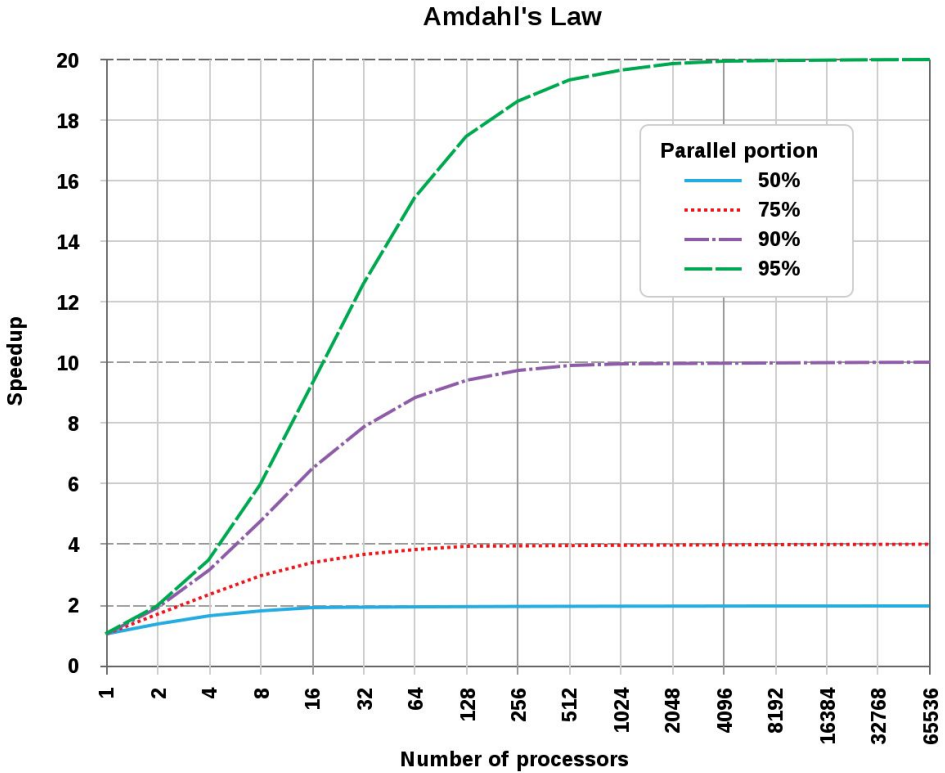
Task: Addition, Multiplication

The same data, different concurrent tasks!

Speeding-up calculations: Moore's law



How much can we speed-up?



[Amdahl's law - Wikipedia]

Amdahl's Law in more details…

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components

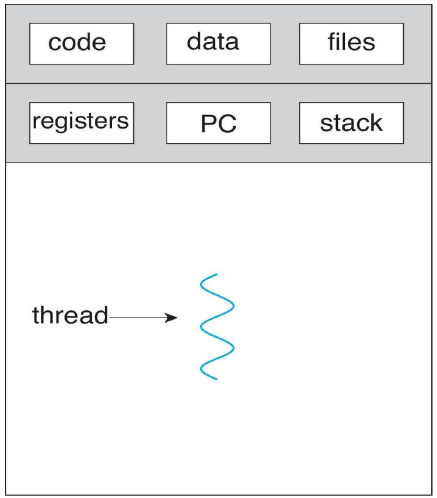
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

  - *S* is fraction of task that is necessarily serial (the rest is parallel)
  - *N* processing cores

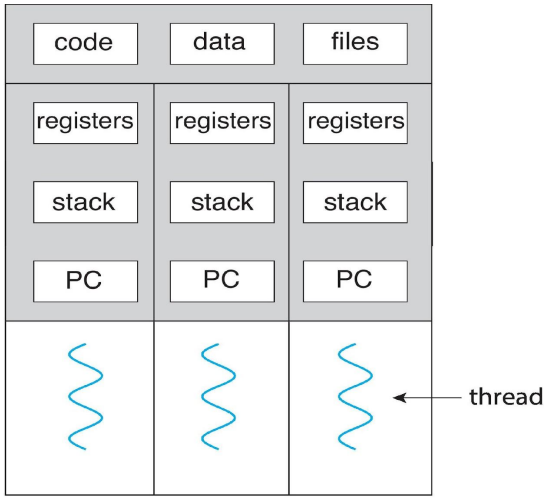
- What is the speedup, if application is 75% parallel and 25% serial, moving from 1 to 2 cores?

- What happens
  - as S approaches 0?
  - as S approaches 1?
  - as N approaches infinity?

How to implement threads?



single-threaded process



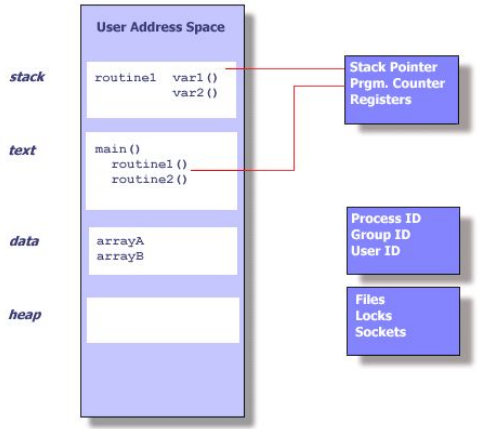
multithreaded process

- A thread is a schedulable execution context
  - Program counter, stack, registers, . . .
- Simple programs use one thread per process
- But can also have multi-threaded programs
  - Multiple threads running in same process's address space

Why need threads: Processes vs Threads

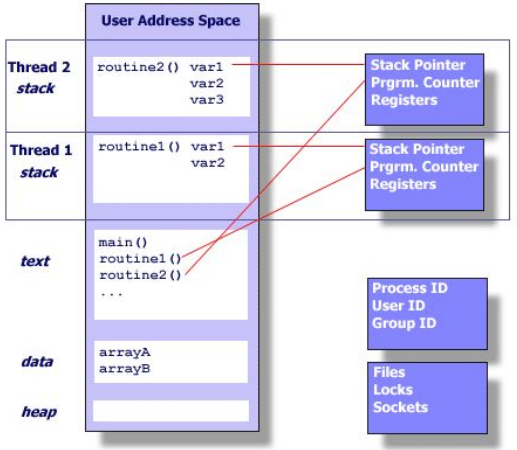
Process

- Each process has its own virtual address space
- **fork() is expensive**



Thread maintains only its own

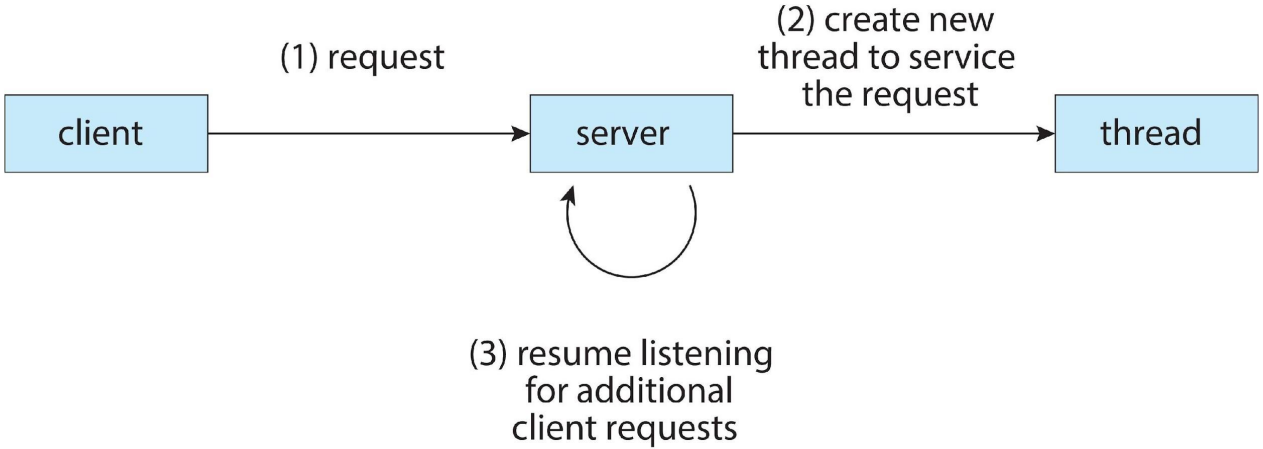
- Stack pointer
- Registers
- Scheduling properties (such as policy or priority)
- Set of pending and blocked signals
- Thread specific data.



More Motivation for threads

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
  - Allows one process to use multiple CPUs or cores
  - Allows program to overlap I/O and computation
- Process creation is heavy-weight while thread creation is light-weight
  - All threads in one process share memory, file descriptors, etc.
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

Example: Multithreaded Server Architecture




```
for (;;) {
        fd = accept_client ();
        thread_create (service_client, &fd);
}
```

Summary of Threading-Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multicore architectures